

Stacks: Theory, Implementation, and Advanced Applications

Introduction to Stacks

A **stack** is a foundational linear data structure in computer science, guided by the **Last-In-First-Out (LIFO)** principle—the last element added is the first removed. Both insertion (**push**) and deletion (**pop**) operate exclusively at one end called the **top** of the stack. Stacks are ubiquitous in programming practice: function call management, undo mechanisms, recursion, backtracking algorithms, browser history, expression evaluation, and more^{[117][114][^102]}.

Key Terminology and Principles

- **LIFO Principle:** Push and pop always affect the top. The last element pushed is the first popped.
- **Top:** The index or node representing the stack's uppermost element.
- **Size:** The current number of elements in the stack.
- **Overflow:** Attempting to push onto a full (fixed-size) stack.
- **Underflow:** Attempting to pop from an empty stack.

Types of Stack Implementations

- **Fixed Size (Static Array):** Stack size declared at creation, can overflow, fast $O(1)$ access but inflexible.
- **Dynamic Size (Linked List):** Stack grows or shrinks as needed, avoids overflow, flexible but with pointer overhead, slower memory access.

Operations and Complexity

Operation	Array Stack	Linked Stack	Description
Push	$O(1)$	$O(1)$	Add element at the top
Pop	$O(1)$	$O(1)$	Remove top element
Peek/Top	$O(1)$	$O(1)$	Retrieve top element
isEmpty	$O(1)$	$O(1)$	Check if stack is empty
isFull	$O(1)$	N/A	Only relevant for array implementation
Size	$O(1)$	$O(1)$	Number of elements

- **Array Stack:** Fixed-size, stored in contiguous memory, fast and simple, but difficult to resize.
- **Linked List Stack:** Dynamic, each node stored in heap with pointer to next, never overflows as long as memory is available^{[103][106][^117]}.

Stack Applications

- **Function and recursion call management** (call stack)
- **Undo/redo in applications**
- **Backtracking and searching algorithms**
- **Browser history (back navigation)**
- **Expression evaluation/conversion** (infix, postfix, prefix)
- **Syntax parsing and validation**
- **Memory management in compilers and OS**^{[114][116][^124]}

Stack Implementation in C: Array-Based

Code Example: Array Stack (with Dynamic Size)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100 // Max stack size

typedef struct {
    int data[MAX];
    int top;
} ArrayStack;

// Initialize stack
void initStack(ArrayStack* s) {
    s->top = -1;
}

// Push operation
int push(ArrayStack* s, int value) {
    if (s->top >= MAX - 1) {
        printf("Stack Overflow!\n");
        return 0;
    }
    s->data[++s->top] = value;
    return 1;
}

// Pop operation
int pop(ArrayStack* s) {
    if (s->top < 0) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return s->data[s->top--];
}

// Peek (Top) operation
int peek(ArrayStack* s) {
    if (s->top < 0) {
```

```

        printf("Stack is empty!\n");
        return -1;
    }
    return s-&gt;data[s-&gt;top];
}

// Display stack
void display(ArrayStack* s) {
    printf("Stack: ");
    for (int i = s-&gt;top; i >= 0; i--)
        printf("%d ", s-&gt;data[i]);
    printf("\n");
}

int main() {
    ArrayStack s;
    initStack(&s);
    push(&s, 10); push(&s, 20); push(&s, 30);
    display(&s); // Output: 30 20 10
    printf("Pop: %d\n", pop(&s)); // Output: 30
    printf("Top: %d\n", peek(&s)); // Output: 20
    display(&s);
    return 0;
}

```

Advanced Notes:

- Array stacks are best for small stacks, real-time systems, embedded use cases.
- Overflow/underflow must be handled robustly.
- Resizing requires creating a new array and copying elements.

Stack Implementation in C: Linked List-Based

Code Example: Linked List Stack

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct {
    Node* top;
    int size;
} LinkedStack;

// Initialize stack
void initStack(LinkedStack* s) {
    s-&gt;top = NULL;
    s-&gt;size = 0;
}

```

```

}

// Push operation
void push(LinkedStack* s, int value) {
    Node* temp = (Node*)malloc(sizeof(Node));
    if (!temp) {
        printf("Memory allocation failed!\n");
        return;
    }
    temp->data = value;
    temp->next = s->top;
    s->top = temp;
    s->size++;
}

// Pop operation
int pop(LinkedStack* s) {
    if (!s->top) {
        printf("Stack Underflow!\n");
        return -1;
    }
    Node* temp = s->top;
    int value = temp->data;
    s->top = temp->next;
    free(temp);
    s->size--;
    return value;
}

// Peek (Top) operation
int peek(LinkedStack* s) {
    if (!s->top) {
        printf("Stack is empty!\n");
        return -1;
    }
    return s->top->data;
}

// Display stack
void display(LinkedStack* s) {
    printf("Stack: ");
    Node* curr = s->top;
    while (curr) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

int main() {
    LinkedStack s;
    initStack(&s);
    push(&s, 40); push(&s, 50); push(&s, 60);
    display(&s); // Output: 60 50 40
    printf("Pop: %d\n", pop(&s)); // Output: 60
    printf("Top: %d\n", peek(&s)); // Output: 50
}

```

```
    display(&s);  
    return 0;  
}
```

Advanced Notes:

- Robust to overflow but may encounter out-of-memory errors; dynamic allocation (no fixed size limit).
- Can efficiently grow/shrink as needed; useful for large or unpredictable stack sizes (e.g., recursion).
- Pointers require extra memory overhead and slightly slower access.

Advanced Stack Applications and Patterns

1. Expression Evaluation & Conversion

- **Infix to Postfix/Prefix evaluation:** Stacks are ideal for converting and evaluating arithmetic expressions using operator precedence ([¹²²][128]).
- **Algorithm:** Scan infix, push operands to output, operators to stack; pop based on precedence; final result in output.

2. Function Call Management (Call Stack)

- Programming languages use stacks to track active function calls, return addresses, and local variables ([¹²⁴][127]).
- Stacks provide fast, automatic memory management and support recursion.

3. Backtracking Algorithms (Maze, Puzzle Solving)

- Stacks manage possible paths or moves, push choices, and pop when backtracking ([¹²³][126]).
- Used in algorithms such as DFS, maze solvers, N-Queen, Sudoku.

4. Parsing and Syntax Validation

- Syntax checking (balanced parentheses, brackets, braces): stack tracks opening symbols, pops when closed ([¹¹⁴][116]).

5. Undo/Redo Mechanisms

- Each action is pushed; undo pops and reverts state ([¹¹⁹][113]).

6. Memory Management and Compiler/OS

- Stacks handle activation records, variable lifetimes, clean up after function return ([¹²⁴][133][136]).

7. Real-World Software Examples

- Browsers: navigating back/forward through history
- Editors: multiple-level undo
- Interpreters: recursive execution calls

Complexity

- **Time Complexity:** All operations $O(1)$
- **Space Complexity:** $O(n)$ (n is number of elements)
- **Array Stack:** $O(1)$ pointer movement
- **Linked Stack:** $O(1)$ pointer manipulation, plus dynamic allocation

High-Level Recommendations

- **Use Array Stack:** For predictable, small stacks with known upper limit
- **Use Linked Stack:** For variable-size stacks or large, unpredictable limits
- Mastering both implementations is essential for advanced algorithm and data structure work, as stacks are ubiquitous in system, language, and application design.

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41]

✱✱

1. https://www.w3schools.com/dsa/dsa_data_stacks.php
2. <https://dev.to/emmanuelayinde/understanding-stack-data-structure-a-step-by-step-guide-to-implementing-stack-in-javascript-3f62>
3. <https://www.ccbp.in/blog/articles/write-a-program-to-implement-stack-using-linked-list-in-c>
4. https://www.reddit.com/r/learnprogramming/comments/1i8z1ox/real_life_uses_of_stacks/
5. <https://getsdready.com/mastering-the-stack-data-structure/>
6. <https://www.geeksforgeeks.org/dsa/implement-stack-using-array/>
7. <https://www.geeksforgeeks.org/dsa/applications-advantages-and-disadvantages-of-stack/>
8. <https://www.geeksforgeeks.org/dsa/introduction-to-stack-data-structure-and-algorithm-tutorials/>
9. <https://takeuforward.org/data-structure/implement-stack-using-linked-list/>
10. <https://www.almabetter.com/bytes/articles/application-of-stack-data-structure>
11. <https://www.geeksforgeeks.org/dsa/stack-data-structure/>
12. <https://iq.opengenus.org/implement-stack-using-array-and-linked-list/>
13. <https://www.youtube.com/watch?v=2YugR6kyOIU>

14. http://gn.dronacharya.info/ECEDept/Downloads/QuestionPapers/4th_sem/DataStructure/unit-2/Lecture3.pdf
15. <https://www.geeksforgeeks.org/dsa/rat-in-a-maze-backtracking-using-stack/>
16. <https://www.geeksforgeeks.org/dsa/stack-vs-heap-memory-allocation/>
17. <https://prepinsta.com/data-structures-and-algorithms-in-python/convert-infix-to-postfix-using-stack-in-python/>
18. <https://www.geeksforgeeks.org/dsa/backtracking-algorithms/>
19. https://en.wikipedia.org/wiki/Call_stack
20. <https://www.geeksforgeeks.org/dsa/convert-infix-expression-to-postfix-expression/>
21. <https://stackoverflow.com/questions/46902791/solve-a-maze-backtracking>
22. <https://stackoverflow.com/questions/72760109/confusion-between-stack-call-stack-and-memory-heap-in-javascript>
23. <https://www.eicta.iitk.ac.in/knowledge-hub/data-structure-with-c/stacks-and-queues-implementation-operations-and-applications>
24. <https://www.geeksforgeeks.org/dsa/evaluation-of-postfix-expression/>
25. <https://www.enjoyalgorithms.com/blog/rat-in-a-maze/>
26. <https://www.spec-india.com/blog/stack-vs-heap-java>
27. <https://www.youtube.com/watch?v=PAceaOSnxQs>
28. <https://heycoach.in/blog/stack-for-backtracking-algorithms/>
29. <https://www.geeksforgeeks.org/operating-systems/introduction-to-stack-memory/>
30. <https://www.slideshare.net/slideshow/unit-2-application-of-stack/251094957>
31. <https://www.youtube.com/watch?v=dWJ1gd5ZHgo>
32. https://www.reddit.com/r/learnjavascript/comments/1gke9we/call_stack_memory_heap_and_storing_variables/
33. https://runestone.academy/ns/books/published/cppds2/linear-basic_infix-prefix-and-postfix-expressions.html
34. <https://www.programiz.com/dsa/stack>
35. https://en.wikipedia.org/wiki/Maze_generation_algorithm
36. <https://www.geeksforgeeks.org/dsa/introduction-to-stack-data-structure-and-algorithm-tutorials/>
37. <https://www.geeksforgeeks.org/dsa/implement-a-stack-using-singly-linked-list/>
38. <https://unacademy.com/blog/application-of-stack-in-data-structure>
39. <https://www.cipherschools.com/blogs/others/understanding-stack-in-data-structures-a-comprehensive-guide/>
40. <https://www.simplilearn.com/tutorials/data-structure-tutorial/stack-implementation-using-linked-list>
41. <https://www.acceldata.io/blog/stack-data-structure-explained-how-lifo-drives-modern-computing>